

File permissions in Linux

Author: Panagiotis Moudouris

Date: September 6th, 2026

Certification: Google Cybersecurity Professional Certificate

Project description

In this project, my objective is to review and manage file and directory permissions within the `/home/researcher2/projects` directory. Using Linux commands, I will identify files with incorrect permissions and modify them to align with the organization's security policies, ensuring that only authorized users have read, write, or execute access.

Check file and directory details

To check the current permissions of all files and directories, including hidden ones, I used the following command: `ls -la`

Based on the output, the current 10-character permission strings for the files and directory are:

```
project_k.txt: -rw-rw-rw-  
project_m.txt: -rw-r-----  
project_r.txt: -rw-rw-r--  
project_t.txt: -rw-rw-r--  
.project_x.txt: -rw--w----  
drafts (directory): drwx--x---
```

Describe the permissions string

Taking the file **project_k.txt** as an example, its permission string is `-rw-rw-rw-`. Here is the breakdown of what each character represents:

1st character (-): Represents the file type. A hyphen indicates it is a regular file. (A `d` would indicate a directory).

2nd to 4th characters (rw-): Represent the **User (owner)** permissions. The `r` stands for read and `w` stands for write. The hyphen (-) means there is no execute permission.

5th to 7th characters (rw-): Represent the **Group** permissions. The group also has read and write permissions, but no execute permission.

8th to 10th characters (rw-): Represent the **Other** (anyone else) permissions. They also have read and write permissions, but no execute permission.

Change file permissions

Based on the current permissions, **project_k.txt** is the only file that allows write access for "others" (`-rw-rw-rw-`). To align with the organization's policy and remove write permissions for others, I used the following command: `chmod o-w project_k.txt`

After running this command, the permissions for `project_k.txt` were updated to `-rw-rw-r--`, meaning "others" now only have read access.

Change file permissions on a hidden file

The hidden file **.project_x.txt** currently has write permissions for the user and group, but no read permissions for the group (`-rw--w----`). Since it is archived, it should not have write permissions for anyone, and both the user and group should have read access. I used the following command to update these permissions: `chmod a-w,g+r .project_x.txt`. This command removes write permissions for all (`a-w`) and adds read permissions for the group (`g+r`). The resulting permission string for the file is `-r--r-----`.

Change directory permissions

The **drafts** directory currently allows execute access for the group (`drwx--x---`). To ensure that only the user (researcher2) has access to this directory and its contents, I removed the group's execute permission using the following command: `chmod g-x drafts`. After applying this command, the permission string for the drafts directory was updated to `drwx-----`, restricting all access solely to the user.

Summary

In this project, I audited the file permissions in the `/home/researcher2/projects` directory to ensure they met the organization's security standards. I used the `ls -la` command to view current permissions, including hidden files. Then, using the `chmod` command, I successfully removed unauthorized write access from regular and hidden files, and restricted directory access to only the authorized user. This process enhanced the overall security and integrity of the project files.